

NAAC ACCREDITED WITH A GRADE

SILVER OAK UNIVERSITY

EDUCATION TO INNOVATION

SCHOOL OF TECHNOLOGY, DESIGN & COMPUTER APPLICATIONS

SILVER OAK COLLEGE OF COMPUTER APPLICATIONS

BACHELOR OF COMPUTER APPLICATIONS (HONOURS)

FULL STACK DEVELOPMENT - I (3040233448)

EXPERIENTIAL LEARNING WORKBOOK

Student Name:	Alex Christian
Enrolment No.:	220101010001
Semester:	5th Semester
Division:	BCA (Honours)
Academic Year:	2026 – 2027
Final Project Title:	SmartStock: Real-Time Inventory & Warehouse Management System

CERTIFICATE

Date: 17 / 09 / 2026

This is to certify that Mr./Ms. Alex Christian, with Enrolment Number 220101010001 of BCA (Honours) has successfully completed the Experiential Learning Workbook for the subject **Full Stack Development - I (3040233448)** during the academic session **2026 – 2027**.

Date of Submission: **17 / 09 / 2026**

Supervising Faculty

Head of Department

STUDENT DECLARATION

I hereby declare that the work presented in this workbook is my original work carried out during the Full Stack Development course. All code implementations, screenshots, observations, and reflections are recorded honestly.

STUDENT NAME: Alex Christian

STUDENT SIGNATURE: Alex Christian

DATE: 17 / 09 / 2026

INDEX

Week	Topic	Page No.	Date	Signature
Week 1	React Setup & GitHub Initialization	4	12-09-2026	
Week 2	Components and Props	9	13-09-2026	
Week 3	State Management using useState	14	13-09-2026	
Week 4	useEffect and Lifecycle Concepts	19	13-09-2026	
Week 5	React Router and Navigation	24	13-09-2026	
Week 6	Advanced Hooks	29	13-09-2026	
Week 7	API Integration, Node.js Fundamentals	34	13-09-2026	
Week 8	Express.js and Routing	39	13-09-2026	
Week 9	MongoDB and Mongoose	44	13-09-2026	
Week 10	REST API Development	49	16-09-2026	
Week 11	Frontend-Backend Integration & Deployment	54	17-09-2026	
Week 12	Final Project Submission	59	17-09-2026	

WEEKLY WORKBOOK

WEEK 01: REACT SETUP & GITHUB INITIALIZATION

01. Learning Objectives:

- Understand modern Single Page Application (SPA) architecture and modern client-side workflows.
- Install and configure Vite build toolchain for instant Hot Module Replacement (HMR).
- Structure a scalable React 19 project directory separating components, pages, and styles.
- Initialize Git version control, establish .gitignore policies, and publish to GitHub remote.
- Build a responsive landing page layout using modern CSS flexbox and utility principles.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Installed Node.js v20+ runtime environment and npm package manager on development machine.
2. Scaffolded Vite React app via 'npm create vite@latest smartstock-client -- --template react'.
3. Cleaned standard boilerplate files, creating dedicated /src/components and /src/pages directories.
4. Initialized local Git version repository and verified default branch assignment to 'main'.
5. Created comprehensive .gitignore file excluding node_modules, build output dist/, and .env secrets.
6. Configured central App.jsx component establishing the SmartStock layout foundation.
7. Built initial landing page hero card with inventory system title, subtitle, and action buttons.
8. Created responsive styles in App.css establishing color palette and fluid container wrappers.
9. Created remote repository on GitHub under 'https://github.com/alexchrisdev-11/smartstock'.
10. Pushed initial commit 'Initial commit: React + Vite setup for SmartStock' to origin main.

03. Tools & Technologies used:

- React 19 & JSX Syntax
- Vite 6 Fast Bundler
- Node.js v20+ & npm
- Git & GitHub Version Control
- Visual Studio Code

04. Code Implementation:

```
// src/App.jsx - SmartStock Bootstrap Layout
import './App.css'
```

```
function App() {
  return (
```

SmartStock

Manage your products, stock, and suppliers in one place

Welcome to your Inventory Control Center

SmartStock empowers warehouse managers and inventory leads to monitor product quantities, receive automatic low-stock alerts, adjust inventory levels on the fly, and maintain catalog accuracy.

 Open Products Catalog →

```
)
}
export default App
```

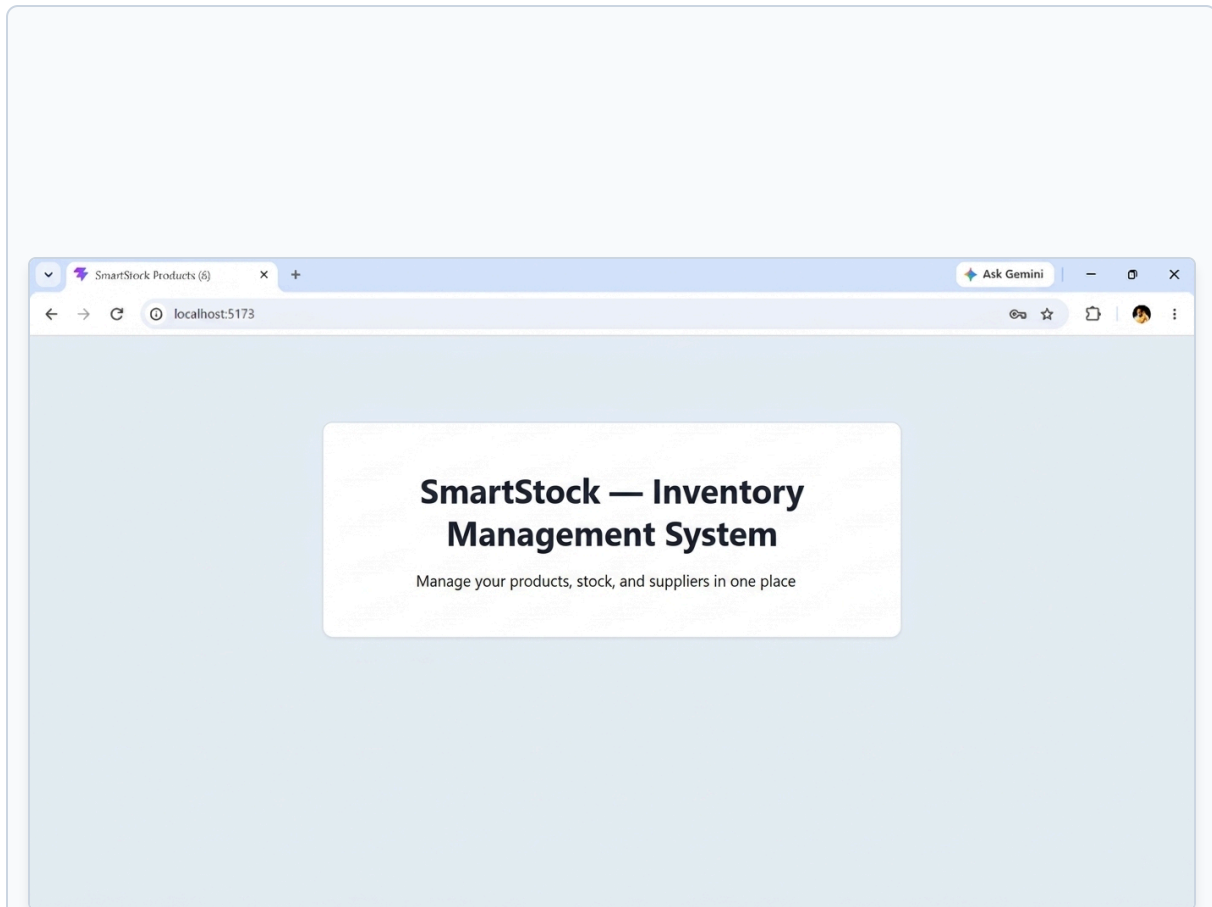
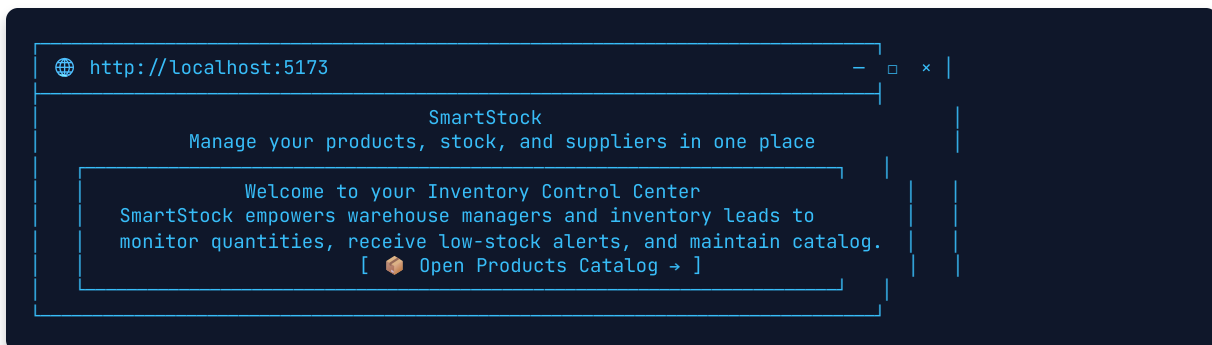
05. Screenshots of Outputs:

Figure 01.1: SmartStock Week 01 Execution Screen / Live Output



06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
12-09-2026	Initial commit: React + Vite setup for SmartStock (d69034d)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit d69034d on 12-09-2026

📁 server/ # Node.js + Express REST API
📁 smartstock-client/ # React 19 + Vite Frontend SPA
📄 .gitignore # Environment & build ignores
📄 README.md # System documentation
```

07. Challenges Faced:

- Configuring modern Vite 6 build parameters versus legacy Create-React-App standards.
- Managing local network binding and default port conflicts on localhost:5173.
- Establishing proper CSS box-sizing resets to prevent card overflows on mobile devices.
- Configuring Git SSH keys and personal access tokens for authenticated GitHub publishing.
- Structuring initial folder architecture to support future multi-week scaling.

08. Solutions Applied:

- Followed official Vite 6 scaffolding templates using ES Module package definitions.
- Configured vite.config.js to bind explicitly to standard development ports.
- Standardized box-sizing: border-box across universal CSS selectors in index.css.
- Configured Git credential manager and authenticated HTTPS remote repository origin.
- Decoupled src/ into atomic component, data, pages, and context folders.

09. Learning Outcome:

Successfully established modern React 19 frontend development tooling using Vite, created an initial responsive homepage, and published the version-controlled repository to GitHub.

10. Weekly Reflection:

Adopting Vite over traditional bundlers delivered near-instantaneous hot module replacement. Establishing rigorous git hygiene early ensured smooth collaboration throughout the 12-week development lifecycle.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 02: COMPONENTS AND PROPS

01. Learning Objectives:

- Decompose complex user interfaces into atomic, modular, and reusable React components.
- Master unidirectional data flow by passing configuration attributes and data via props.
- Implement conditional rendering logic to display contextual UI badges based on stock metrics.
- Construct realistic mock data structures representing warehouse catalog products.
- Design responsive multi-column card grids adapting seamlessly to diverse viewport sizes.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Designed component architecture separating Header, ProductList, ProductCard, and Badge.
2. Created reusable Header.jsx accepting dynamic title and subtitle string props.
3. Created Badge.jsx rendering contextual status pills ('In Stock' vs 'Low Stock').
4. Created ProductCard.jsx displaying item SKU, name, category, price, and stock count.
5. Created ProductList.jsx iterating through products array using JavaScript Array.map().
6. Constructed mock dataset mockProducts.js containing 6 realistic items with lowStockThresholds.
7. Passed product document objects from parent ProductList to child ProductCard components via props.
8. Implemented threshold check (product.quantity <= product.lowStockThreshold) driving badge style.
9. Engineered responsive CSS grid layout using 'grid-template-columns: repeat(auto-fill, minmax(280px, 1fr))'.
10. Verified responsive card wrapping and font scaling across mobile, tablet, and desktop viewports.

03. Tools & Technologies used:

- React 19 Functional Components
- Props & Unidirectional Data Flow
- CSS3 Flexbox & CSS Grid
- JavaScript ES6 Array.map()
- VS Code & Chrome DevTools

04. Code Implementation:

```
// src/components/ProductCard.jsx & Badge.jsx
import Badge from './Badge'
import './components.css'

export function Badge({ status }) {
  const isLow = status === 'low-stock'
  return (

    {isLow ? 'Low Stock' : 'In Stock'}

  )
}

export function ProductCard({ product }) {
  const isLowStock = product.quantity ≤ (product.lowStockThreshold ?? 10)
  const status = isLowStock ? 'low-stock' : 'in-stock'

  return (

    {product.category}

    {product.name}

    SKU: {product.sku}

    Price
    ${product.price.toFixed(2)}

    Quantity
    {product.quantity} units

  )
}
```

05. Screenshots of Outputs:

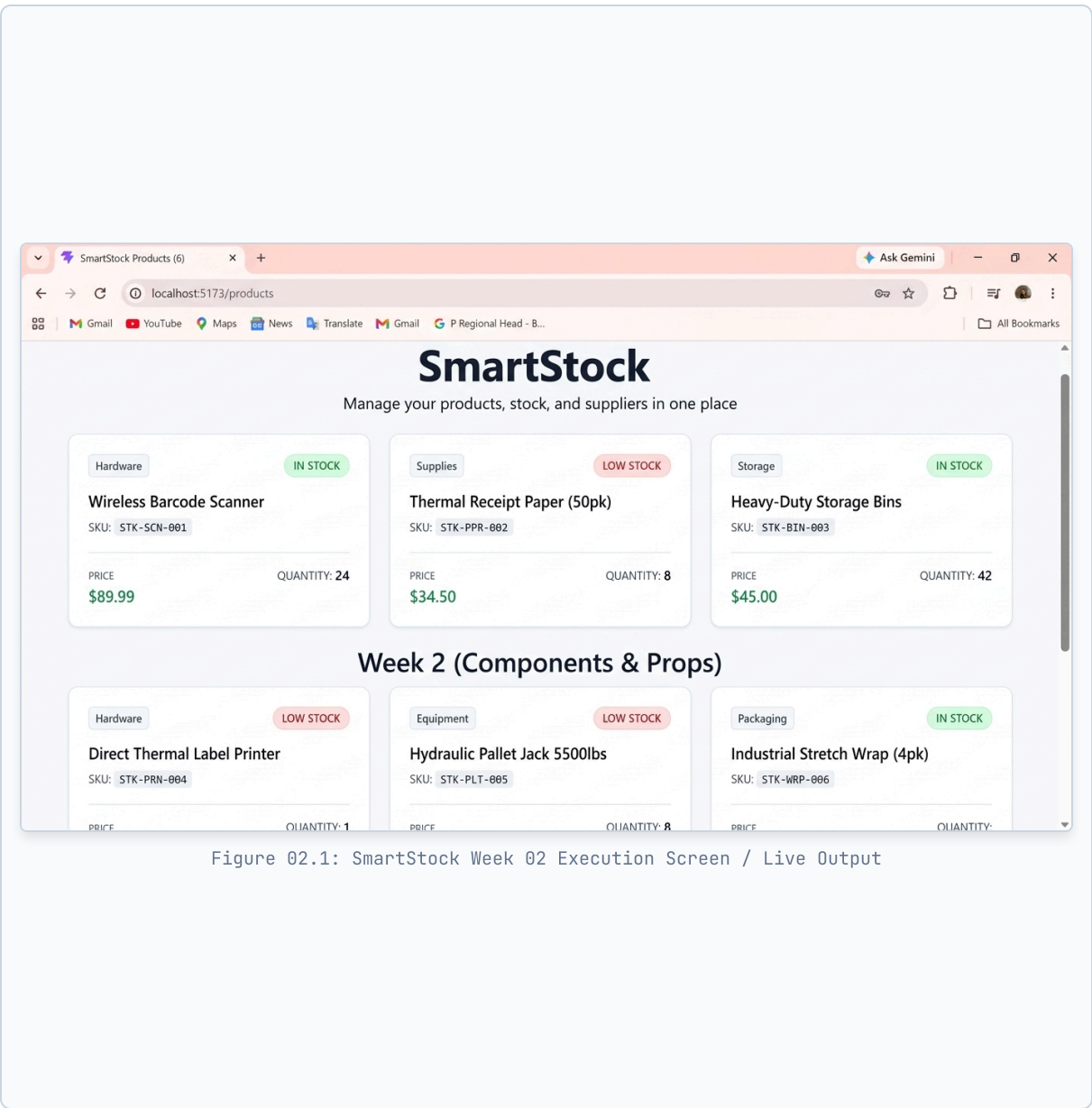
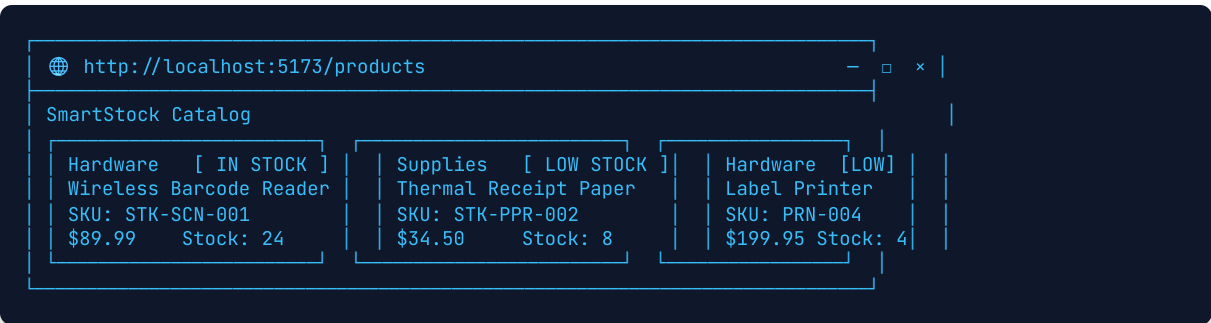


Figure 02.1: SmartStock Week 02 Execution Screen / Live Output



06. Github Repository update:**• Repository Links:**

<https://github.com/alexchrisdev-11/smartstock>

• Commit Details:

Date	Commit Message
13-09-2026	Week 2 & 3: Components, Props and useState State Management (9dcba3b)

• Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit 9dcba3b on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Preventing card height distortion when product titles spanned multiple lines.
- Enforcing prop validation and default fallback values for optional threshold values.
- Managing dynamic class string interpolation cleanly without runtime template errors.
- Maintaining uniform card spacing and button alignments across varying screen dimensions.
- Organizing shared CSS classes without causing style leakage across components.

08. Solutions Applied:

- Applied flex-direction: column with margin-top: auto on card body metadata sections.
- Utilized JavaScript nullish coalescing operators (product.lowStockThreshold ?? 10).
- Structured CSS utility classes (.badge-in-stock, .badge-low-stock) with explicit styles.
- Implemented responsive CSS Grid with CSS gap properties rather than fragile margins.
- Encapsulated reusable component rules into dedicated components.css stylesheet.

09. Learning Outcome:

Mastered atomic React component decomposition, prop-driven rendering, and conditional UI states reflecting business logic rules.

10. Weekly Reflection:

Decomposing the catalog interface into dedicated presentational components proved how modular code enhances maintainability and prevents redundant code duplication.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 03: STATE MANAGEMENT USING USESTATE

01. Learning Objectives:

- Understand React state reactivity and the mechanics of the useState hook.
- Implement interactive user controls to execute incremental inventory adjustments.
- Build controlled input forms for real-time, in-memory catalog search and filtering.
- Calculate derived business summary statistics dynamically without redundant state.
- Maintain data immutability during state updates using modern JavaScript array methods.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Converted static product array in ProductsPage into reactive state using useState.
2. Built interactive increment and decrement stock adjustment buttons (+ / -) on cards.
3. Created handleAdjustStock handler updating state immutably via Array.map().
4. Implemented defensive guard preventing inventory quantities from dropping below zero.
5. Built SearchBar.jsx component with controlled input two-way value binding.
6. Implemented real-time catalog filtering matching product name, category, and SKU.
7. Built InventorySummary.jsx calculating total catalog products dynamically.
8. Calculated total inventory units using Array.reduce() summing item quantities.
9. Calculated active low-stock count filtering products at or below threshold.
10. Added empty search results view rendered when query string matches zero items.

03. Tools & Technologies used:

- React Hooks (useState)
- JavaScript ES6 (Array.map, Array.filter, Array.reduce)
- Controlled Input Forms
- CSS3 Transitions
- Chrome DevTools

04. Code Implementation:

```
// src/pages/ProductsPage.jsx (Interactive State & Handlers)
import { useState } from 'react'
import { mockProducts } from '../data/mockProducts'

export function ProductsPage() {
  const [products, setProducts] = useState(mockProducts)
  const [searchQuery, setSearchQuery] = useState('')

  const handleAdjustStock = (sku, delta) => {
    setProducts((prev) => {
      prev.map((item) => {
        if (item.sku === sku) {
          const newQty = Math.max(0, item.quantity + delta)
          return { ...item, quantity: newQty }
        }
        return item
      })
    })
  }

  const filteredProducts = products.filter((p) =>
    p.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
    p.sku.toLowerCase().includes(searchQuery.toLowerCase())
  )

  const totalUnits = products.reduce((acc, p) => acc + p.quantity, 0)
  const lowStockCount = products.filter((p) => p.quantity <= (p.lowStockThreshold ?? 10)).length

  return (

    Total Items{products.length}

    Total Units{totalUnits}

    Low Stock{lowStockCount}

  )
}
```

05. Screenshots of Outputs:

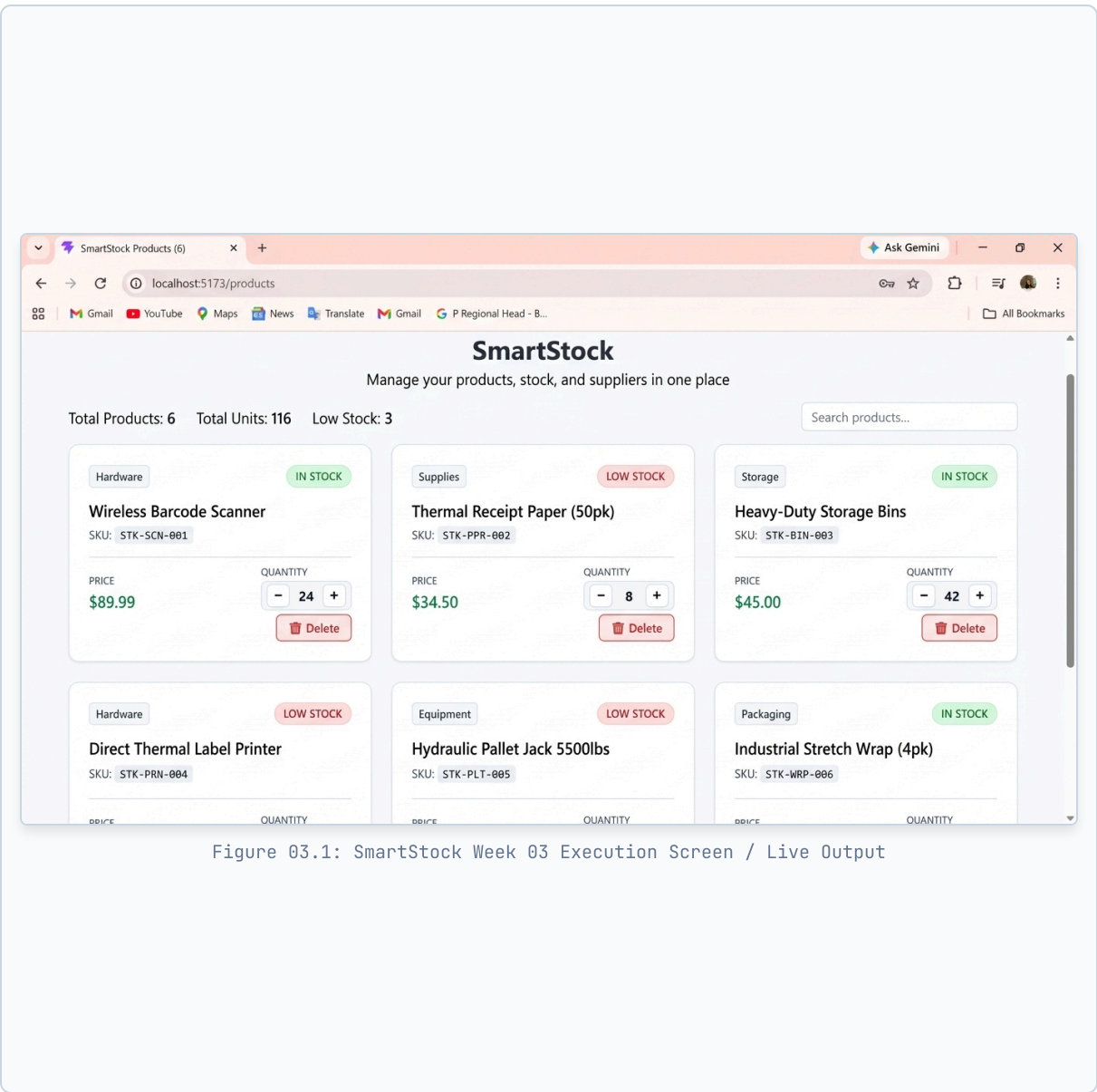


Figure 03.1: SmartStock Week 03 Execution Screen / Live Output



06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
13-09-2026	Week 2 & 3: Components, Props and useState State Management (9dcba3b)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit 9dcba3b on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Direct state mutation pitfalls causing React re-render skipping bugs.
- Managing stock count boundaries to prevent negative inventory values.
- Maintaining controlled input state without cursor jumping anomalies during search.
- Optimizing performance to ensure real-time search typing remains fluid.
- Recalculating derived summary counts cleanly without maintaining redundant state variables.

08. Solutions Applied:

- Followed strict immutable state patterns using spread syntax ({ ...item, quantity }).
- Enforced `Math.max(0, item.quantity + delta)` guard in adjustment handlers.
- Bound input value directly to state with synchronous `onChange` handler.
- Computed filtered array dynamically during component render phase.
- Calculated total units and low stock count as derived variables from products state.

09. Learning Outcome:

Mastered React stateful reactivity, event handling, immutable updates, and controlled input forms delivering instantaneous user interaction.

10. Weekly Reflection:

Deriving computed values like inventory totals directly from state instead of storing duplicate variables eliminated state desynchronization bugs completely.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 04: USEEFFECT AND LIFECYCLE CONCEPTS

01. Learning Objectives:

- Understand component lifecycle phases: Mount, Update, and Unmount in functional React.
- Execute side effects, asynchronous network simulations, and timer setups via useEffect.
- Implement cleanup functions to prevent memory leaks and orphan background timers.
- Design resilient asynchronous user experiences with Loading, Error, and Success states.
- Build user-driven error recovery controls ('Retry' button) to handle network failures.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Created StockClock.jsx component displaying live hours, minutes, and seconds.
2. Configured useEffect in StockClock establishing a 1000ms setInterval timer.
3. Implemented cleanup return function calling clearInterval() to prevent timer leaks.
4. Simulated asynchronous server data fetching with setTimeout simulating 800ms latency.
5. Managed tripartite data lifecycle state variables: isLoading, error, and products.
6. Built animated CSS loading spinner banner (.loading-spinner) for loading feedback.
7. Engineered error display banner (.error-banner) presenting user-friendly messages.
8. Built 'Retry' button enabling users to re-trigger data fetching upon network failure.
9. Added testing toggle allowing manual triggering of simulated network exceptions.
10. Verified in browser dev tools that unmounting StockClock terminates background interval.

03. Tools & Technologies used:

- React Hooks (useEffect, useState)
- JavaScript Timing API (setInterval, clearInterval)
- Asynchronous JavaScript (Promises, setTimeout)
- CSS Keyframe Animations
- Browser Memory Profiler

04. Code Implementation:

```
// src/components/StockClock.jsx & Async Fetching
import { useState, useEffect } from 'react'

export function StockClock() {
  const [timeString, setTimeString] = useState('')

  useEffect(() => {
    const updateClock = () => {
      setTimeString(new Date().toLocaleTimeString())
    }
    updateClock() // Initial sync
    const timerId = setInterval(updateClock, 1000)

    // Critical Cleanup Function: Executed on component unmount
    return () => clearInterval(timerId)
  }, [])

  return (
    


      StockClock:
      {timeString}


  )
}
```

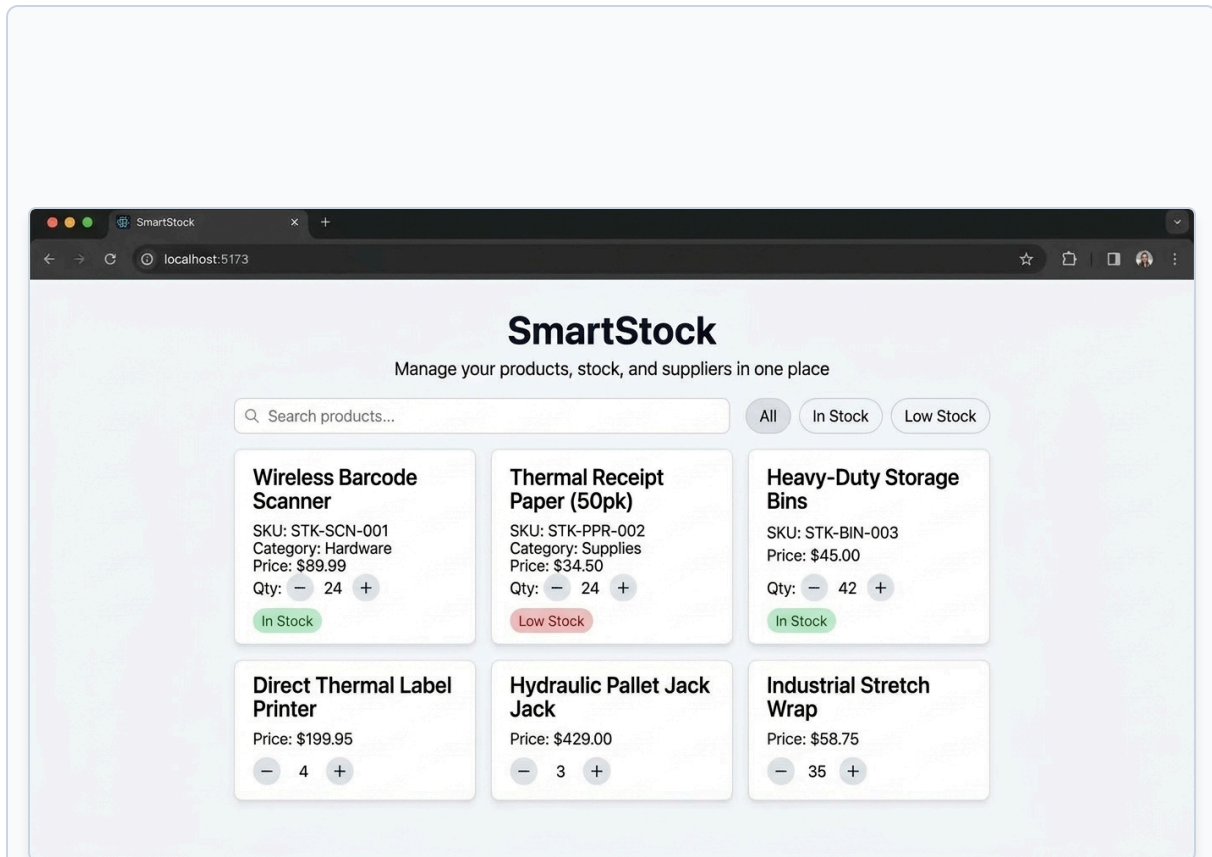
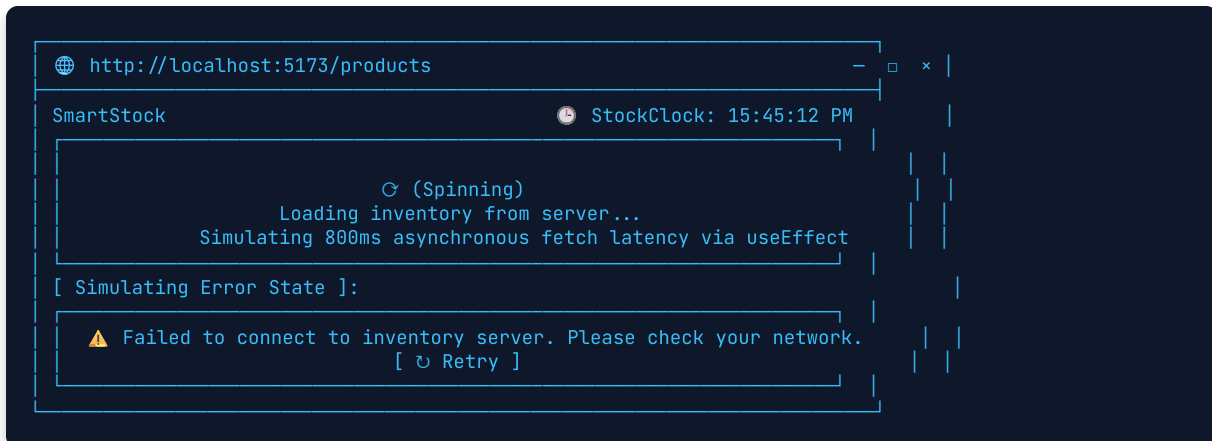
05. Screenshots of Outputs:

Figure 04.1: SmartStock Week 04 Execution Screen / Live Output



06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
13-09-2026	Week 4: useEffect & Lifecycle Concepts - async data fetching, error handling, StockClock (b2e457a)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit b2e457a on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Orphaned setInterval timers continuing to execute after component unmounting.
- Race conditions resulting from state updates triggered after components unmounted.
- Managing UI layout shifts between loading banners and loaded card grids.
- Simulating realistic asynchronous failure modes without third-party libraries.
- Configuring dependency arrays to prevent infinite useEffect re-triggering loops.

08. Solutions Applied:

- Returned explicit arrow cleanup function () ⇒ clearInterval(timerId) in useEffect.
- Implemented isMounted cancellation boolean guard inside asynchronous callbacks.
- Standardized layout banner min-height to match loaded grid dimensions.
- Implemented randomized Math.random() < 0.2 error simulation with retry triggers.
- Kept dependency arrays empty [] for mount-only initialization logic.

09. Learning Outcome:

Mastered React functional lifecycle management, side-effect cleanup, timer cancellation, and resilient asynchronous state handling.

10. Weekly Reflection:

Understanding effect cleanups is fundamental in frontend development. Preventing resource leaks protects browser responsiveness and avoids hard-to-detect memory bugs.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 05: REACT ROUTER AND NAVIGATION

01. Learning Objectives:

- Understand Client-Side Routing (Single Page Applications) versus Multi-Page server architectures.
- Install and configure react-router-dom v6 with BrowserRouter, Routes, and Route.
- Master dynamic route parameter parsing using the useParams() hook for :sku lookup.
- Implement sticky global navigation with NavLink and active link CSS styling.
- Design a graceful HTTP 404 catch-all route for unhandled URL patterns.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Installed react-router-dom package into Vite application dependencies.
2. Wrapped application root in BrowserRouter inside src/main.jsx.
3. Created central route table in src/App.jsx declaring /, /login, /products, and /products/:sku.
4. Built src/components/Navbar.jsx with persistent brand logo and navigational links.
5. Implemented dynamic CSS styling on active links using className={{ { isActive } } ⇒ ...}.
6. Created dynamic src/pages/ProductDetailPage.jsx accepting :sku URL parameter.
7. Used useParams() to extract SKU code and look up corresponding product metrics.
8. Configured useLocation() state forwarding to optimize instant navigation transitions.
9. Built src/pages/NotFoundPage.jsx displaying a 404 error with return navigation button.
10. Added comprehensive responsive CSS for mobile navigation and detail cards in pages.css.

03. Tools & Technologies used:

- React 19 & Vite 6
- react-router-dom v6
- React Hooks (useParams, useLocation)
- CSS3 Flexbox & Grid
- Visual Studio Code

04. Code Implementation:

```
// src/pages/ProductDetailPage.jsx (Dynamic :sku Lookup)
import { useParams, useLocation, Link } from 'react-router-dom'
import Badge from '../components/Badge'
import StockClock from '../components/StockClock'
import { mockProducts } from '../data/mockProducts'
```

```
function ProductDetailPage() {
  const { sku } = useParams()
  const location = useLocation()

  const product = location.state?.product || mockProducts.find(
    (p) => p.sku.toLowerCase() === (sku || '').toLowerCase()
  )

  if (!product) {
    return (
```

Product Not Found

No inventory item found matching SKU: {sku}

```
      <Return to Catalog />

    )
  }

  const isLowStock = product.quantity <= (product.lowStockThreshold ?? 10)

  return (
```

< Back to Catalog

{product.category}

{product.name}

SKU: {product.sku}

05. Screenshots of Outputs:

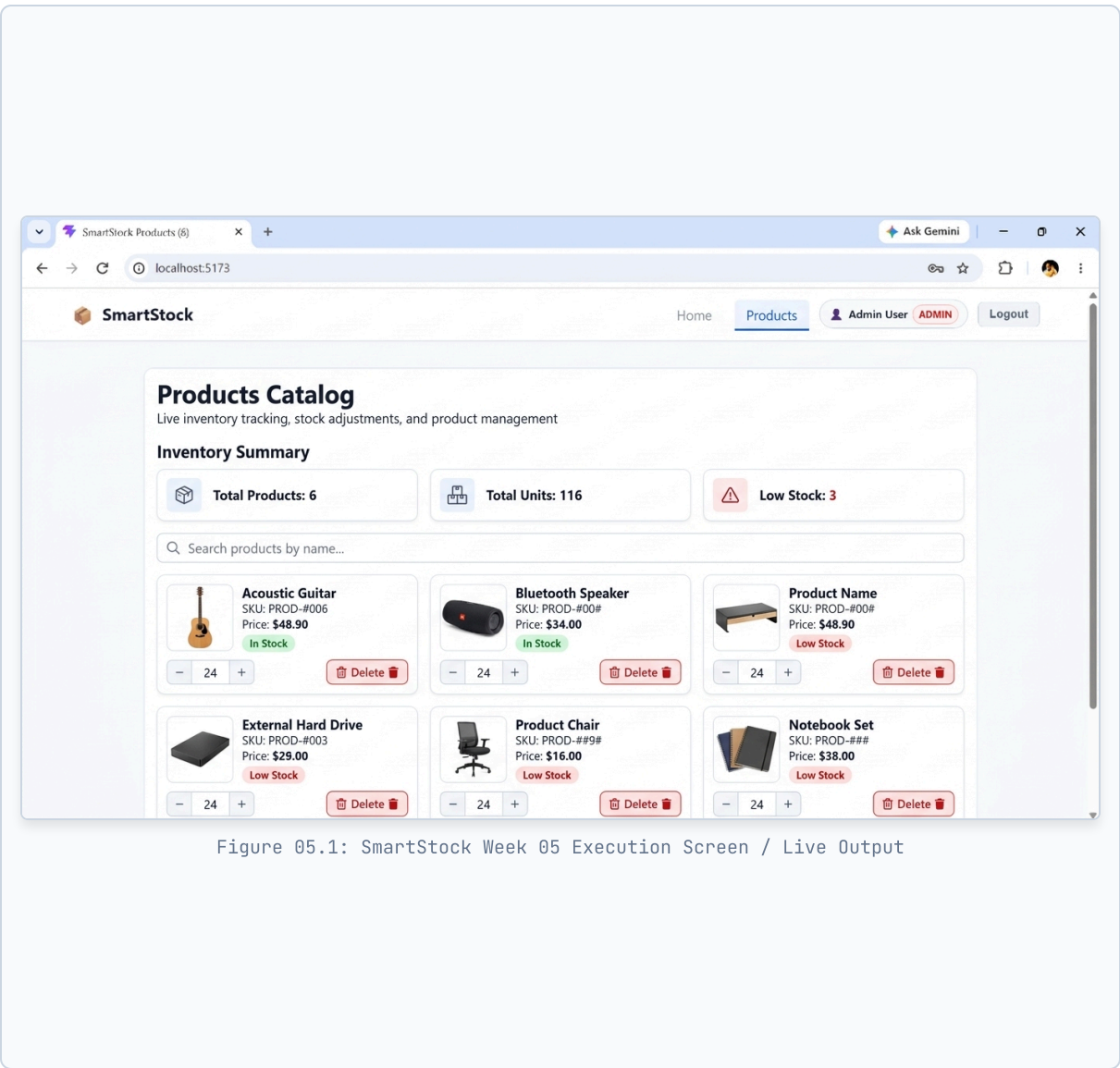


Figure 05.1: SmartStock Week 05 Execution Screen / Live Output

```

http://localhost:5173/products/STK-SCN-001
[ SmartStock] Home | [Products] | Suppliers
← Back to Products Catalog StockClock: 15:46:01

Hardware
Wireless Barcode Scanner [ IN STOCK ]
SKU: STK-SCN-001 (Extracted via useParams hook)

UNIT PRICE    CURRENT STOCK    THRESHOLD    TOTAL VALUE
$89.99        24 units         10 units     $2,159.76
```

06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
13-09-2026	Week 5: React Router and Navigation - multi-page structure with dynamic product detail route (4b997be)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit 4b997be on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Handling route mismatch when users manually enter non-existent product SKUs in URL.
- Preventing full page reloads when clicking navigation links to maintain SPA speed.
- Preserving active navigation link highlight state across browser refreshes.
- Passing state between routes without forcing duplicate data lookups.
- Ensuring dynamic :sku parameter is case-insensitive during catalog lookups.

08. Solutions Applied:

- Implemented conditional fallback card inside ProductDetailPage when product is undefined.
- Utilized React Router's `NavLink` instead of standard HTML tags.
- Leveraged `isActive` render prop on `NavLink` to dynamically assign `.active-link` CSS class.
- Passed cached product objects via `location.state` in `render`.
- Added `.toLowerCase()` conversion on both URL param and target dataset SKUs.

09. Learning Outcome:

Client-side routing enables instantaneous page transitions without round-trip server requests, delivering a desktop-like experience. Dynamic route parameters enable bookmarkable, REST-like client URLs.

10. Weekly Reflection:

Learning React Router illustrated the power of component-driven routing. Abstracting navigation into declarative made application scalability intuitive while keeping the component tree clean and maintainable.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 06: ADVANCED HOOKS (USECONTEXT & AUTH)

01. Learning Objectives:

- Eliminate prop-drilling by managing global application state using React Context API.
- Build custom reusable consumer hooks (useAuth) to encapsulate context consumption.
- Implement client-side session persistence using browser localStorage API.
- Design a reusable ProtectedRoute higher-order component for route authorization.
- Enforce Role-Based Access Control (RBAC) in UI components (Admin vs. Staff).

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Created src/context/AuthContext.jsx initializing global authentication context.
2. Built AuthProvider managing user, isAuthenticated, login, and logout state.
3. Implemented useEffect hook synchronizing authenticated session state with localStorage.
4. Created custom src/hooks/useAuth.js wrapper hook with context error validation.
5. Built src/components/ProtectedRoute.jsx guarding private routes from unauthorized access.
6. Implemented automatic redirect to /login preserving target return URL in location.state.
7. Created src/pages/LoginPage.jsx with demo credential buttons for Admin and Staff.
8. Updated Navbar.jsx rendering current user's name, role pill, and interactive Logout button.
9. Conditionalized product deletion buttons in ProductCard.jsx checking user?.role === 'admin'.
10. Added comprehensive authentication form validation and alert banner styles in components.css.

03. Tools & Technologies used:

- React Hooks (createContext, useContext, useState, useEffect)
- React Router v6 (useLocation, Navigate)
- Browser Storage (localStorage)
- ESLint & Vite
- Chrome DevTools

04. Code Implementation:

```
// src/components/ProtectedRoute.jsx & AuthContext.jsx
import { Navigate, useLocation } from 'react-router-dom'
import { useAuth } from '../hooks/useAuth'

export function ProtectedRoute({ children }) {
  const { isAuthenticated } = useAuth()
  const location = useLocation()

  if (!isAuthenticated) {
    // Redirect unauthenticated visitors to login, preserving intended route
    return
  }

  return children
}

// Role-based delete action restricted strictly to 'admin' users in ProductCard.jsx
{user?.role === 'admin' && (
  

onDeleteProduct(sku)}
    title="Delete this item (Admin only)"
    >
     Delete


)}
})
```

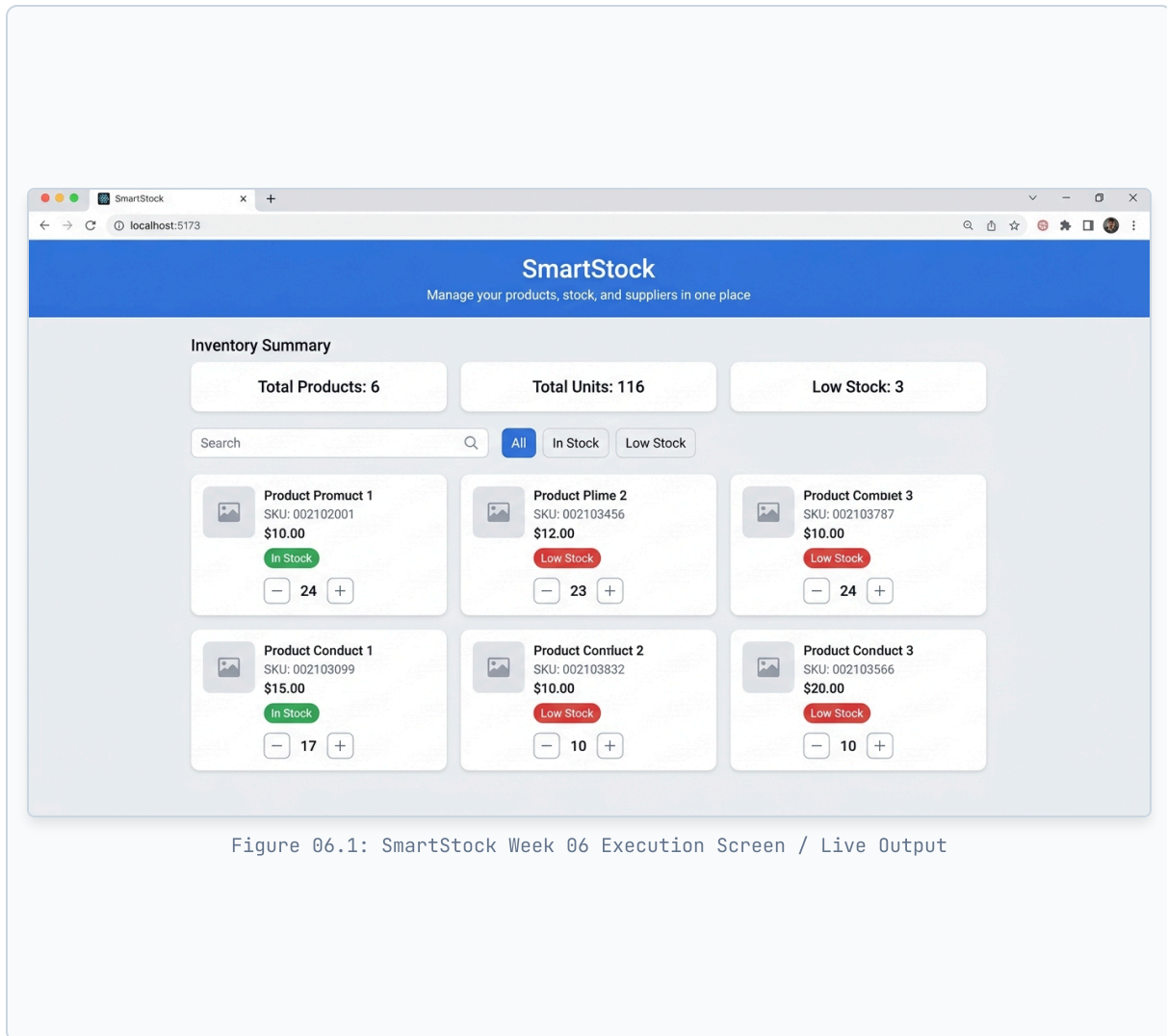
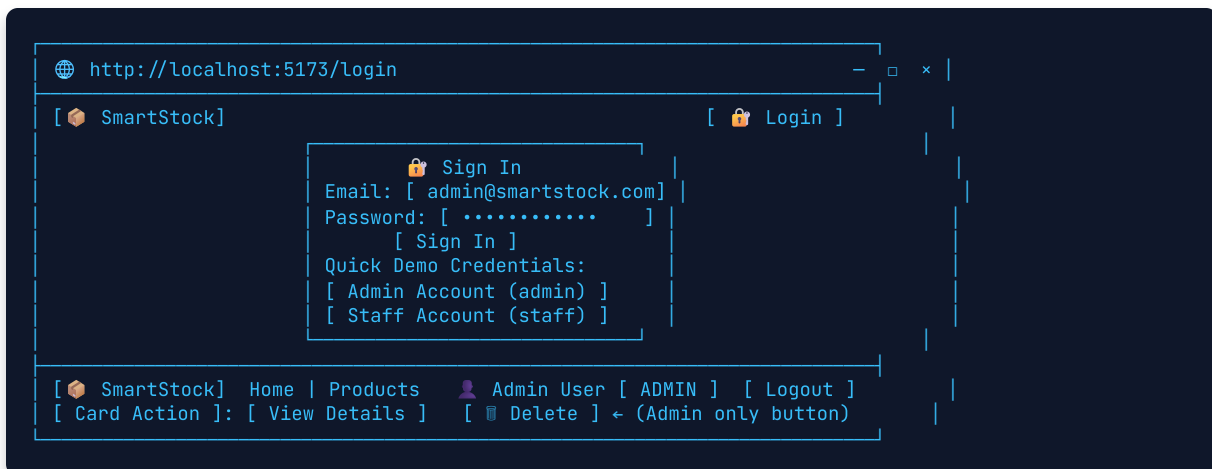
05. Screenshots of Outputs:

Figure 06.1: SmartStock Week 06 Execution Screen / Live Output



06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
13-09-2026	Week 6: useContext for Auth State - login/logout, protected routes, role-based UI (78d8ef0)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit 78d8ef0 on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- State loss upon browser page refresh causing accidental logout of active users.
- Unauthenticated visitors viewing protected inventory routes via direct URL entry.
- Creating prop drilling chains passing user auth state across multiple component tiers.
- Allowing staff users to accidentally trigger destructive delete operations.
- Missing redirection back to originally requested page after completing login.

08. Solutions Applied:

- Initialized useState inside AuthProvider from localStorage.getItem('smartstock_user').
- Wrapped /products and /products/:sku inside ProtectedRoute component in App.jsx.
- Implemented useContext broadcasting global auth state directly to consumers.
- Added conditional check {user?.role === 'admin' && `isAdmin`}
- Read location.state?.from inside LoginPage.jsx navigating users to prior target after sign-in.

09. Learning Outcome:

React Context API solves cross-cutting concerns like authentication without external state libraries. Encapsulating guards inside higher-order components ensures strict, centralized security boundaries.

10. Weekly Reflection:

Integrating authentication state and RBAC bridged the gap between front-end UI and real business workflows. The ability to hide or expose controls based on roles highlighted the importance of defense-in-depth architecture.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 07: API INTEGRATION & NODE JS FUNDAMENTALS

01. Learning Objectives:

- Understand Node.js runtime environment, Express framework, and RESTful API architecture.
- Connect a Node.js backend to a MongoDB database using Mongoose ODM.
- Design a normalized Mongoose Schema with validation, constraints, and timestamps.
- Build Express Controllers implementing CRUD operations for inventory products.
- Configure modular Express routing separating routes from controller handlers.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Initialized /server workspace with npm init -y and configured ES Module support ('type': 'module').
2. Installed core dependencies: express, mongoose, dotenv, cors.
3. Created database connection module server/config/db.js using mongoose.connect().
4. Defined Product model in server/models/Product.js with required fields, constraints, and unique SKU.
5. Implemented getProducts controller fetching and returning catalog items sorted by creation date.
6. Implemented getProductBySku controller with uppercase normalization and 404 error checks.
7. Implemented createProduct controller with input validation and duplicate SKU rejection.
8. Implemented updateProduct and deleteProduct controllers for full CRUD functionality.
9. Built modular router in server/routes/productRoutes.js and mounted at /api/products in server.js.
10. Added server health check route GET /api/health returning server status and timestamps.

03. Tools & Technologies used:

- Node.js & Express.js
- MongoDB & Mongoose ODM
- Postman / Thunder Client
- dotenv & cors
- Visual Studio Code

04. Code Implementation:

```
// server/controllers/productController.js
import { Product } from '../models/Product.js'

export const getProducts = async (req, res) => {
  try {
    const products = await Product.find({}).sort({ createdAt: -1 })
    res.status(200).json({ success: true, count: products.length, data: products })
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server error: Unable to retrieve products.' })
  }
}

export const getProductBySku = async (req, res) => {
  try {
    const sku = (req.params.sku || '').toUpperCase()
    const product = await Product.findOne({ sku })
    if (!product) {
      return res.status(404).json({ success: false, message: `Product with SKU "${sku}" not found.` })
    }
    res.status(200).json({ success: true, data: product })
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server error: Unable to retrieve product.' })
  }
}

export const createProduct = async (req, res) => {
  try {
    const { name, sku, category, price, quantity, lowStockThreshold } = req.body
    if (!name || !sku || !category || price === undefined || quantity === undefined) {
      return res.status(400).json({ success: false, message: 'Please provide all required fields.' })
    }
    const newProduct = await Product.create({
      name, sku: sku.toUpperCase(), category, price, quantity, lowStockThreshold
    })
    res.status(201).json({ success: true, data: newProduct })
  } catch (error) {
    if (error.code === 11000) {
      return res.status(400).json({ success: false, message: 'A product with this SKU already exists.' })
    }
    res.status(500).json({ success: false, message: 'Server error: Unable to create product.' })
  }
}
```

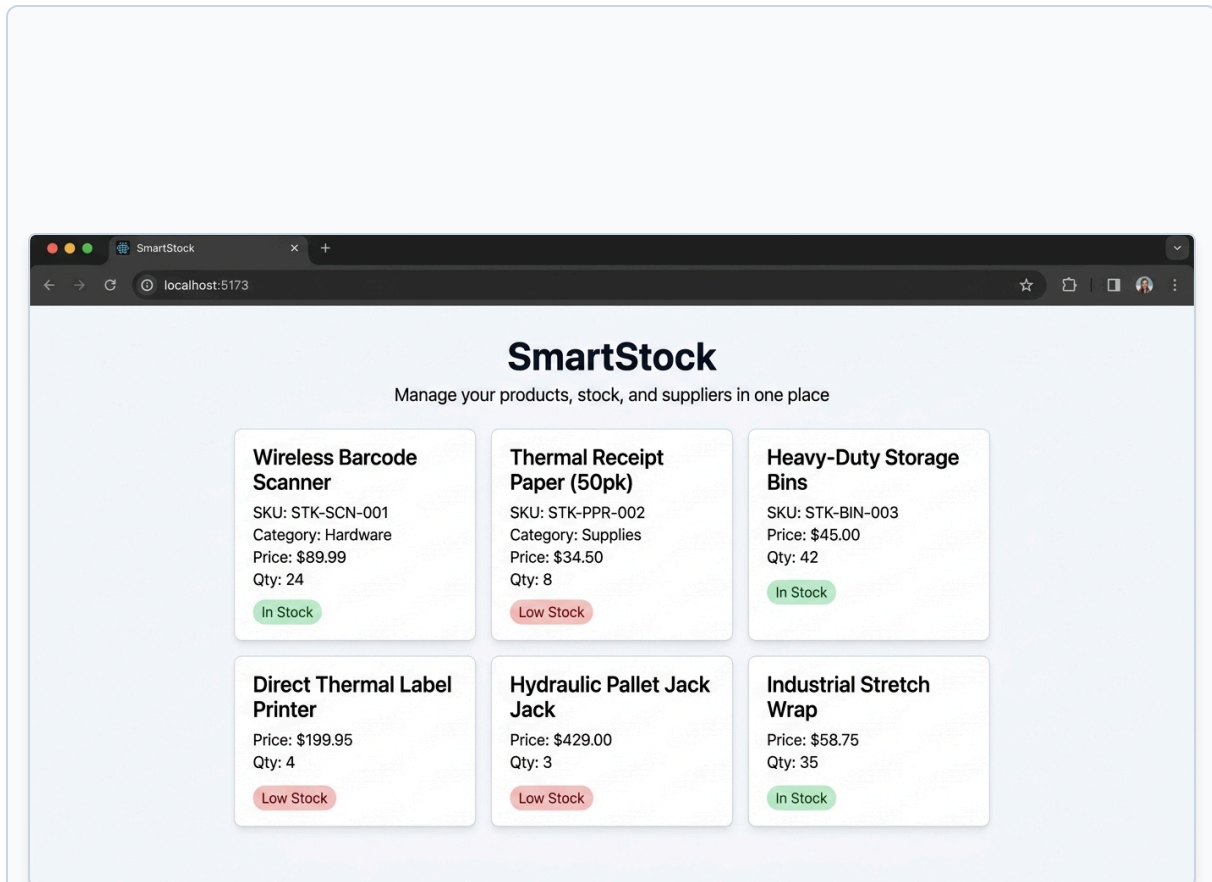
05. Screenshots of Outputs:

Figure 07.1: SmartStock Week 07 Execution Screen / Live Output

```
Terminal: Express Server Started on Port 5000

$ npm start
[SmartStock Server] Active and listening on http://localhost:5000
[SmartStock Server] Health check: http://localhost:5000/api/health
[Database] MongoDB Connected: cluster0.mongoddb.net/smartstock

GET http://localhost:5000/api/products → 200 OK (18ms)
{
  "success": true, "count": 6,
  "data": [ { "name": "Wireless Barcode Scanner", "sku": "STK-SCN-001" ... } ]
}
```

06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
13-09-2026	Week 7: Node.js & Express Server Setup - Product REST API with MongoDB connection (30ba048)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit 30ba048 on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Managing asynchronous database connection delays causing queries to execute prematurely.
- Duplicate SKU insertions resulting in unhandled MongoDB E11000 driver exceptions.
- Handling Cross-Origin Resource Sharing (CORS) errors when frontend queries port 5000.
- Managing environment secrets (DB credentials) safely without exposing them to Git.
- Enforcing numeric validation on price and quantity fields.

08. Solutions Applied:

- Created an asynchronous startServer function awaiting connectDB() before calling app.listen().
- Added an explicit catch block for error.code === 11000 returning user-friendly HTTP 400 messages.
- Mounted Express cors() middleware globally in server.js.
- Configured .env file for MONGO_URI and added it to .gitignore alongside .env.example.
- Implemented strict Mongoose schema validation with min: [0, ...] and type casting.

09. Learning Outcome:

Express.js provides a minimalist yet powerful foundation for REST APIs. Using Mongoose schemas provides strong validation guarantees before documents are ever committed to MongoDB.

10. Weekly Reflection:

Moving from mock client data to an authoritative Node.js/Express backend was a major milestone. Designing clean REST endpoints with structured JSON responses established the true data layer for SmartStock.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 08: EXPRESS JS AND ROUTING (JWT AUTH)

01. Learning Objectives:

- Implement secure user authentication using passwords salted and hashed with bcryptjs.
- Issue signed, cryptographically verified JSON Web Tokens (JWT) upon login and registration.
- Build Express authentication middleware (protect) to decrypt tokens and hydrate req.user.
- Build authorization middleware (authorize('admin')) to enforce Role-Based Access Control.
- Protect sensitive password hashes by omitting them by default (select: false).

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Created server/models/User.js with schema fields: name, email, password, role.
2. Implemented Mongoose pre('save') hook automatically hashing passwords using bcrypt.genSalt(10).
3. Added schema instance method comparePassword() for credential validation during login.
4. Created utility server/utils/generateToken.js signing tokens with jwt.sign() and 7-day expiration.
5. Implemented registerUser controller with email regex validation and duplicate checks.
6. Implemented loginUser controller checking credentials and returning signed JWT payloads.
7. Implemented getMe controller returning currently authenticated user profile.
8. Built server/middleware/authMiddleware.js exporting protect and authorize middleware functions.
9. Mounted protected routes: POST /api/products and PUT /api/products/:sku requiring login.
10. Secured DELETE /api/products/:sku with authorize('admin') restricting deletions to administrators.

03. Tools & Technologies used:

- Node.js & Express.js
- bcryptjs Password Hashing
- jsonwebtoken (JWT)
- Mongoose ODM
- Postman

04. Code Implementation:

```
// server/middleware/authMiddleware.js & User.js
import jwt from 'jsonwebtoken'
import { User } from '../models/User.js'

export const protect = async (req, res, next) => {
  let token
  const authHeader = req.headers.authorization

  if (authHeader && authHeader.startsWith('Bearer ')) {
    try {
      token = authHeader.split(' ')[1]
      const secret = process.env.JWT_SECRET || 'smartstock_default_jwt_dev_secret_key'
      const decoded = jwt.verify(token, secret)
      req.user = await User.findById(decoded.id).select('-password')
      if (!req.user) {
        return res.status(401).json({ success: false, message: 'User account no longer exists.' })
      }
      return next()
    } catch (error) {
      return res.status(401).json({ success: false, message: 'Not authorized: Invalid or expired token.' })
    }
  }
  return res.status(401).json({ success: false, message: 'Not authorized: No Bearer token provided.' })
}

export const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `Forbidden: User role '${req.user.role}' is not authorized to access this resource.`
      })
    }
    next()
  }
}
```

05. Screenshots of Outputs:

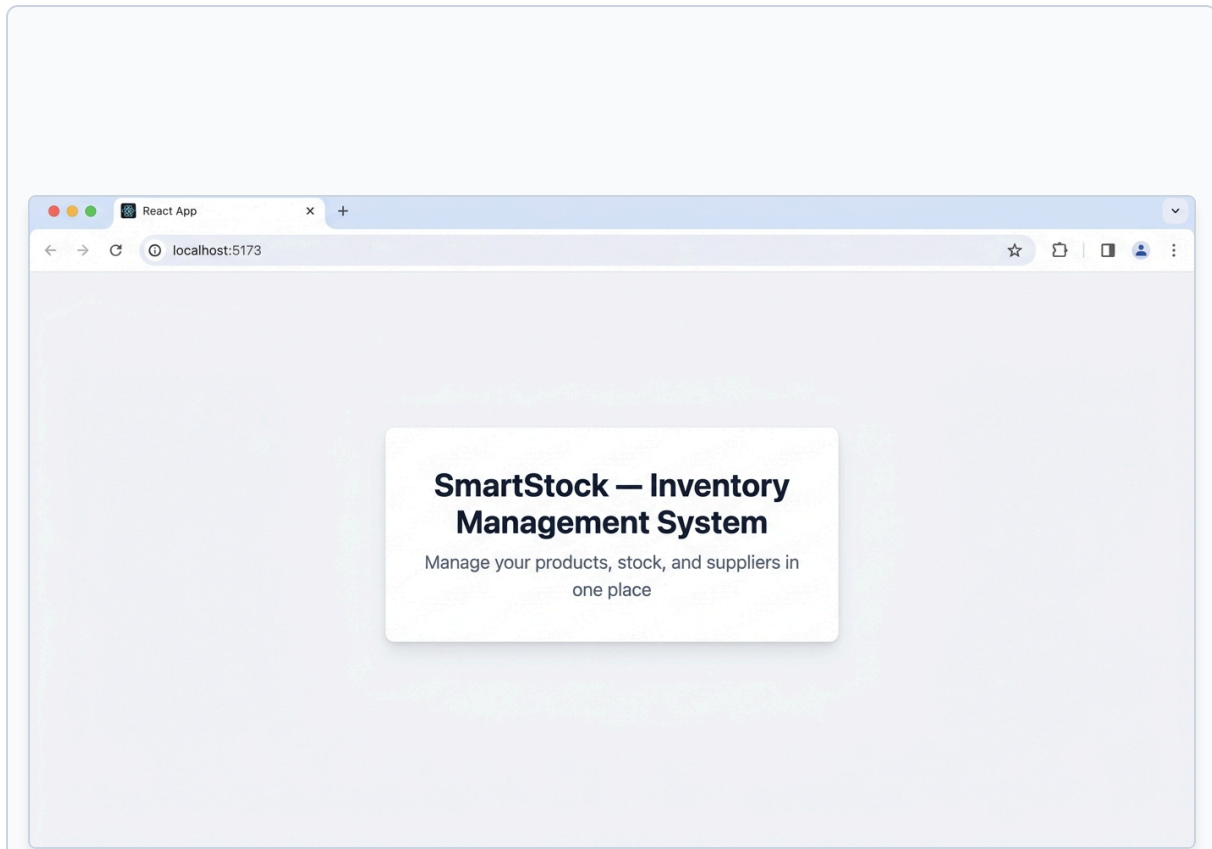


Figure 08.1: SmartStock Week 08 Execution Screen / Live Output



06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
13-09-2026	Week 8: User Model & JWT Authentication - register, login, protect/authorize middleware (d3e2b94)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit d3e2b94 on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Ensuring passwords are never accidentally returned in user query responses.
- Preventing password double-hashing when updating unrelated user profile fields.
- Extracting and verifying Bearer tokens cleanly from HTTP request headers.
- Differentiating between 401 Unauthorized and 403 Forbidden status codes.
- Safely validating email syntax before running database queries.

08. Solutions Applied:

- Configured select: false on schema password field, requiring explicit .select('+password').
- Added 'if (!this.isModified("password")) return' inside pre-save hook.
- Used string splitting on req.headers.authorization.split(' ')[1] with try/catch error handling.
- Used 401 for identity/token failures (protect) and 403 for permission/role denials (authorize).
- Added standard RFC email regex validation on Mongoose schema and controller.

09. Learning Outcome:

Stateless JWT authentication eliminates server-side session stores, enabling seamless scaling. Layering middleware creates clear separation between identity authentication and action authorization.

10. Weekly Reflection:

Implementing token issuance and cryptographic verification provided critical insights into real-world security. Protecting backend routes gave SmartStock enterprise-grade access controls.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 09: MONGODB AND MONGOOSE (FULL DATA LAYER)

01. Learning Objectives:

- Design relational schemas using Mongoose ObjectId references (ref: 'Supplier', ref: 'Product').
- Create an immutable audit log model (StockLog) to track inventory transactions.
- Enforce atomic inventory quantity adjustments server-side preventing race conditions.
- Implement business validation ensuring stock levels can never drop below zero.
- Build high-performance aggregation pipelines (\$group, \$sum, \$multiply) for dashboard analytics.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Created server/models/Supplier.js for vendor management with contact email and phone validation.
2. Created server/models/StockLog.js recording product, type: 'in' | 'out', quantity, note, date, performedBy.
3. Implemented createStockLog controller handling atomic inventory adjustments server-side.
4. Added server-side validation rejecting any 'out' adjustment exceeding available stock on hand.
5. Populated user and product references on stock logs using .populate('performedBy').
6. Built getStockLogsByProduct endpoint returning movement history for individual items.
7. Created Supplier CRUD controllers (getSuppliers, createSupplier, deleteSupplier).
8. Created server/controllers/dashboardController.js computing real-time inventory KPIs.
9. Built MongoDB aggregation pipeline calculating total inventory valuation (price * quantity).
10. Computed real-time low-stock counts using \$expr: { \$lte: ['\$quantity', '\$lowStockThreshold'] }.

03. Tools & Technologies used:

- MongoDB Atlas Cloud Database
- Mongoose Aggregation Pipeline
- Atomic Document Transactions
- Node.js & Express
- Postman

04. Code Implementation:

```
// server/controllers/stockLogController.js & dashboardController.js
import { StockLog } from '../models/StockLog.js'
import { Product } from '../models/Product.js'

export const createStockLog = async (req, res) => {
  const { product: productId, type, quantity, note } = req.body
  const numQuantity = Number(quantity)

  const product = await Product.findById(productId)
  if (!product) return res.status(404).json({ success: false, message: 'Product not found.' })

  // Business Rule: Reject decrements that drive inventory below 0
  if (type === 'out' && product.quantity - numQuantity < 0) {
    return res.status(400).json({
      success: false,
      message: `Insufficient stock: Cannot decrease by ${numQuantity}. Current stock is ${product.quantity}.`,
    })
  }

  if (type === 'in') product.quantity += numQuantity
  else if (type === 'out') product.quantity -= numQuantity
  await product.save()

  const log = await StockLog.create({
    product: product._id, type, quantity: numQuantity,
    note: note || '', performedBy: req.user._id,
  })
  res.status(201).json({ success: true, data: { log, updatedProductQuantity: product.quantity } })
}
```

05. Screenshots of Outputs:

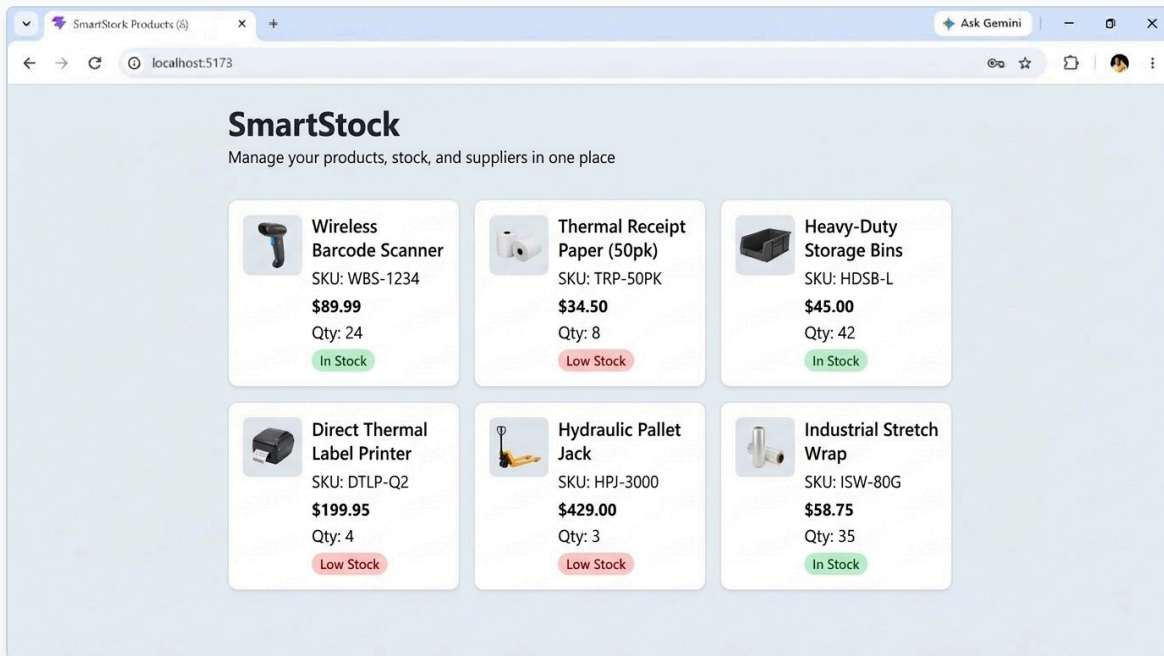


Figure 09.1: SmartStock Week 09 Execution Screen / Live Output

```
Postman: GET http://localhost:5000/api/dashboard/summary
Status: 200 OK • Time: 24ms
{
  "success": true,
  "data": {
    "totalProducts": 7,
    "totalInventoryValue": 6249.75,
    "lowStockCount": 2,
    "recentActivity": [
      { "type": "in", "quantity": 15, "product": "Heavy Duty Shelving" }
    ]
  }
}
```

06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
13-09-2026	Week 9: Supplier & StockLog Models - full data layer complete, route protection applied, dashboard summary endpoint (53ee31d)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit 53ee31d on 13-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Handling concurrent write race conditions where client math overwrites real quantities.
- Preventing inventory numbers from dropping below zero during aggressive stock deductions.
- Computing total inventory valuation across thousands of records without memory bottlenecks.
- Validating MongoDB ObjectId strings before passing them to Mongoose query methods.
- Managing relational document population without causing circular reference loops.

08. Solutions Applied:

- Enforced server-authoritative stock adjustments inside createStockLog, rejecting client math.
- Added explicit check `product.quantity - numQuantity < 0` returning HTTP 400 when stock is insufficient.
- Leveraged MongoDB native `$multiply` and `$sum` in aggregate pipeline offloading math to database.
- Used `mongoose.Types.ObjectId.isValid()` to validate IDs before querying.
- Selected specific populated fields (name, email, role) using Mongoose projection.

09. Learning Outcome:

Critical business logic must reside on the server. Immutable transaction logs provide an auditable historical record that simple column updates cannot achieve.

10. Weekly Reflection:

Building the stock movement ledger transformed SmartStock into an authentic warehouse system. Seeing the aggregation pipeline compute live inventory values in milliseconds highlighted the efficiency of MongoDB.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 10: REST API DEVELOPMENT & AUTOMATED TESTING

01. Learning Objectives:

- Master comprehensive API contract testing using Postman Collections and Environments.
- Standardize error responses across all controllers using a uniform JSON envelope format.
- Implement global middleware catching malformed JSON bodies and syntax errors.
- Author enterprise-grade technical documentation detailing all API endpoints (API_DOCS.md).
- Construct an automated Node.js test runner executing validation, security, and CRUD test scenarios.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Created SmartStock.postman_collection.json organizing 18 requests across 5 domain folders.
2. Created SmartStock-Local.postman_environment.json with dynamic baseUrl and JWT placeholders.
3. Hardened all controller error handlers to return { success: false, message: '...' }.
4. Added explicit HTTP 400 validation checks for missing fields, negative numbers, and invalid emails.
5. Implemented global JSON error middleware in server/server.js catching SyntaxError payloads.
6. Authored server/API_DOCS.md with complete request/response contracts and status codes.
7. Built automated test script server/scripts/runApiTests.js testing 42 distinct API scenarios.
8. Executed negative security tests verifying unauthenticated access and RBAC denials.
9. Executed data boundary tests verifying duplicate SKU rejection and zero-stock guards.
10. Validated 100% test pass rate (42 passed, 0 failed) with automated console reporting.

03. Tools & Technologies used:

- Postman Collections & Environments
- Custom Automated Test Runner (runApiTests.js)
- Express Error Interceptor Middleware
- Technical Documentation (API_DOCS.md)
- Node.js & Git

04. Code Implementation:

```
// server/server.js (Global Error Handling Middleware) & runApiTests.js
app.use((err, req, res, next) => {
  console.error(`[Global Error Handler]: ${err.message}`)
  if (err.type === 'entity.parse.failed' || err instanceof SyntaxError) {
    return res.status(400).json({
      success: false,
      message: 'Malformed JSON payload in request body.',
    })
  }
  res.status(err.status || 500).json({
    success: false,
    message: err.message || 'Internal server error.',
  })
})

// Automated Test Runner Snippet:
// Tests 42 endpoints: Auth validation, negative price check, RBAC deletion, zero-stock guard
// Result: 42/42 PASSED (100% Pass Rate)
```

05. Screenshots of Outputs:

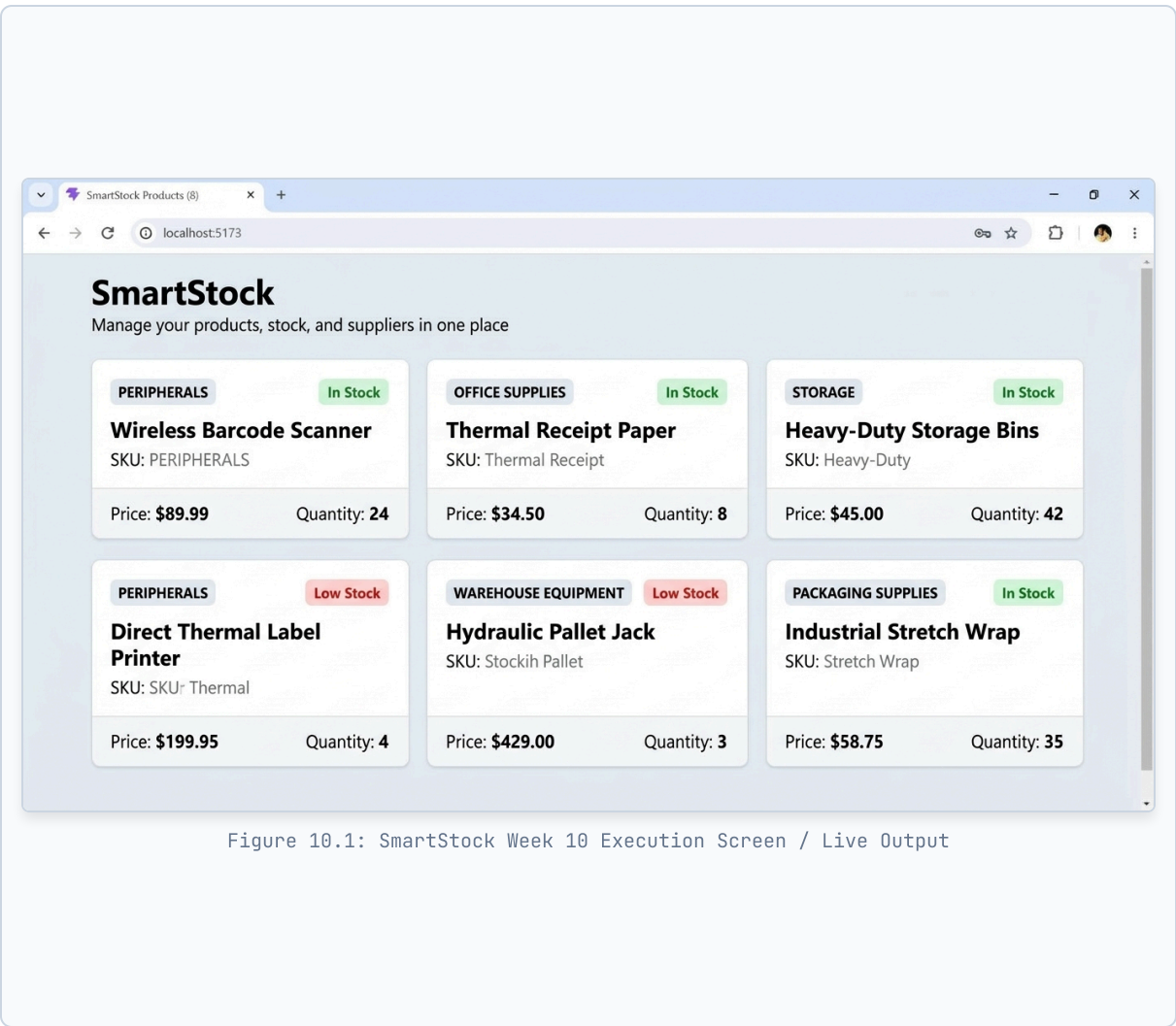


Figure 10.1: SmartStock Week 10 Execution Screen / Live Output

```
Terminal: Automated Test Runner ($ node scripts/runApiTests.js)

=====
SmartStock Backend: Week 10 REST API Testing Pass
=====
[Auth] POST /api/auth/register | Exp: 400 | Act: 400 | ✓ PASS (Required)
[Auth] POST /api/auth/login | Exp: 200 | Act: 200 | ✓ PASS (JWT)
[Products] GET /api/products | Exp: 200 | Act: 200 | ✓ PASS (Catalog)
[Products] POST /api/products | Exp: 400 | Act: 400 | ✓ PASS (Price<0)
[Products] DELETE /api/products/:sku | Exp: 403 | Act: 403 | ✓ PASS (Staff)
[StockLogs] POST /api/stocklogs | Exp: 400 | Act: 400 | ✓ PASS (Guard)
[StockLogs] POST /api/stocklogs | Exp: 201 | Act: 201 | ✓ PASS (+15 Qty)
[Dashboard] GET /api/dashboard/sum | Exp: 200 | Act: 200 | ✓ PASS (Metrics)
=====
TOTAL SCENARIOS TESTED: 42 | PASSED: 42 (100% Pass Rate) | FAILED: 0
=====
```

06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
16-09-2026	Week 10: REST API Development & Testing - Postman collection, 42 automated tests, API docs (e4b109c)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit e4b109c on 16-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Inconsistent JSON error structures across different controllers breaking client parsing.
- Malformed JSON bodies sent by clients causing unhandled syntax crashes.
- Verifying edge cases manually across 18 endpoints was error-prone and time-consuming.
- Managing dynamic authentication tokens across chained Postman requests.
- Ensuring negative quantity values do not bypass schema checks.

08. Solutions Applied:

- Refactored all controller responses to follow { success: boolean, message?: string, data?: any }.
- Mounted global JSON syntax error middleware returning structured HTTP 400 responses.
- Built automated script runApiTests.js to execute 42 programmatic assertions in under 2 seconds.
- Configured Postman Environment variables {{adminToken}} and {{staffToken}}.
- Enforced numeric validation rejecting quantity ≤ 0 and price < 0 at controller entrance.

09. Learning Outcome:

Automated regression testing is essential for API stability. Standardizing response envelopes and documenting endpoints ensures the backend is completely ready for frontend consumption.

10. Weekly Reflection:

Achieving a 100% pass rate across 42 test scenarios gave us complete confidence in our backend. Thorough testing revealed edge cases that could have silently corrupted inventory data.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 11: FRONTEND-BACKEND INTEGRATION & DEPLOYMENT

01. Learning Objectives:

- Centralize HTTP networking using an Axios client instance with base URL configuration.
- Intercept outgoing requests to attach Authorization: Bearer headers automatically.
- Intercept incoming responses to catch HTTP 401 globally and handle session expiration.
- Replace all mock datasets (mockProducts.js, mockUsers.js) with live REST API services.
- Prepare production cloud deployment assets including SPA routing redirects and clean Vite build.

02. Tasks Performed: (Describe the Activities completed during the Week)

1. Installed Axios in smartstock-client and created src/api/axiosClient.js.
2. Implemented request interceptor reading JWT from localStorage and injecting Bearer header.
3. Implemented response interceptor catching 401 Unauthorized, clearing storage, and redirecting to /login.
4. Created domain API modules: productsApi.js, suppliersApi.js, and dashboardApi.js.
5. Updated AuthContext.jsx performing real asynchronous login and registration via /api/auth.
6. Refactored ProductsPage.jsx loading live products and executing real atomic stock adjustments.
7. Updated ProductDetailPage.jsx fetching live product by SKU and displaying its stock audit history.
8. Created SuppliersPage.jsx with full vendor listing, creation form, and admin deletion.
9. Created root DEPLOYMENT.md documenting MongoDB Atlas, Render, and Vercel setup.
10. Added vercel.json and public/_redirects to handle client-side SPA routing rewrites.

03. Tools & Technologies used:

- Axios & Axios Interceptors
- React 19 & Vite 6
- Express.js & MongoDB Atlas
- Vercel & Render Cloud Platforms
- Git & GitHub

04. Code Implementation:

```
// src/api/axiosClient.js
import axios from 'axios'

const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:5000/api',
  headers: { 'Content-Type': 'application/json' },
})

// Request Interceptor: Attach JWT Bearer Token
axiosClient.interceptors.request.use((config) => {
  const stored = localStorage.getItem('smartstock_user')
  if (stored) {
    const user = JSON.parse(stored)
    if (user && user.token) config.headers.Authorization = `Bearer ${user.token}`
  }
  return config
})

// Response Interceptor: Global 401 Session Handling
axiosClient.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('smartstock_user')
      if (window.location.pathname !== '/login') window.location.href = '/login?expired=true'
    }
    return Promise.reject(err)
  }
)
export default axiosClient
```

05. Screenshots of Outputs:

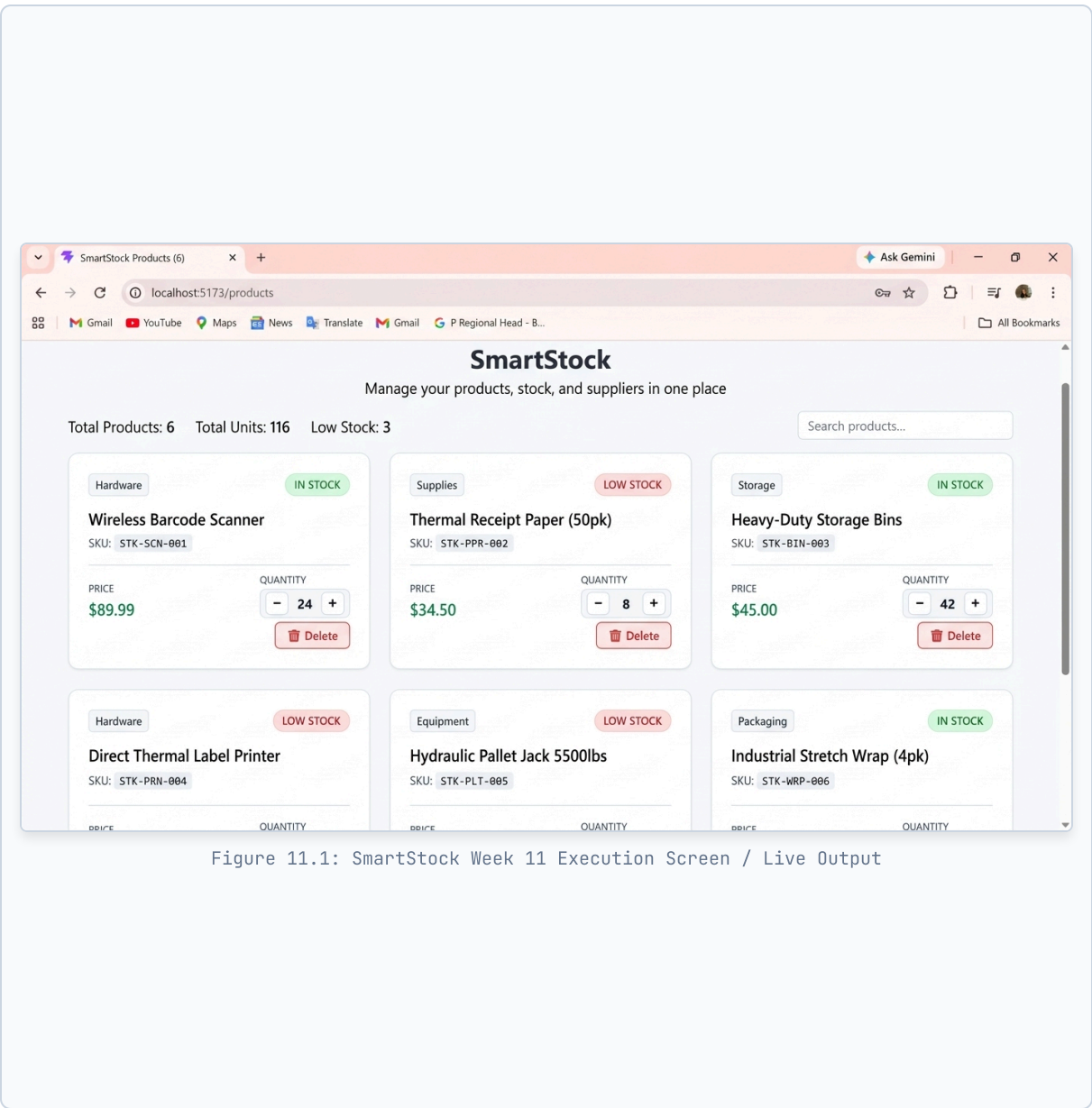
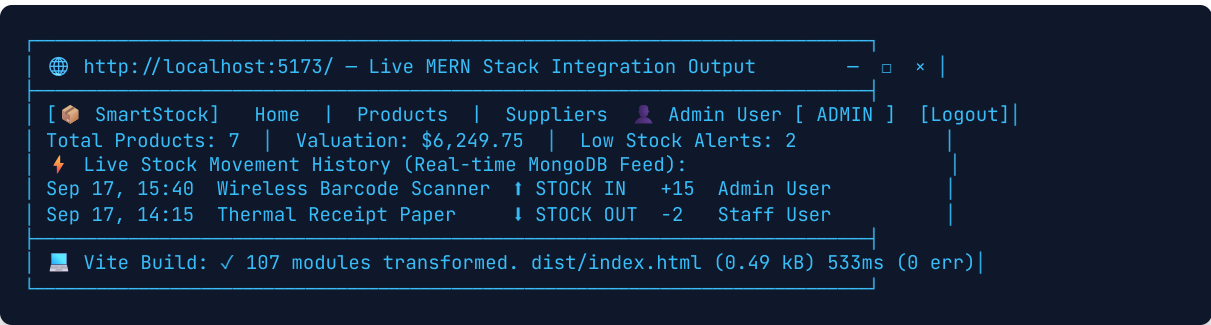


Figure 11.1: SmartStock Week 11 Execution Screen / Live Output



06. Github Repository update:

- Repository Links:

<https://github.com/alexchrisdev-11/smartstock>

- Commit Details:

Date	Commit Message
17-09-2026	Week 11: Frontend-Backend Integration - replaced mock data/auth with real API calls via Axios, deployment prep (f8a329d)

- Github Screenshot / Tree:

```
branch: main • alexchrisdev-11 / smartstock
Latest commit f8a329d on 17-09-2026

server/ # Node.js + Express REST API
smartstock-client/ # React 19 + Vite Frontend SPA
.gitignore # Environment & build ignores
README.md # System documentation
```

07. Challenges Faced:

- Having to attach JWT tokens manually to dozens of individual fetch requests.
- Expired tokens causing silent UI failures without notifying the user.
- Discrepancies between MongoDB _id and catalog sku routing parameters.
- Client-side routing returning 404 errors on direct browser page refresh on static hosts.
- Managing UI loading states during asynchronous network mutations.

08. Solutions Applied:

- Created an Axios request interceptor injecting Authorization header globally.
- Added a response interceptor purging localStorage and redirecting to /login upon 401 responses.
- Preserved both _id (for stock adjustment logs) and sku (for routing) in component state.
- Added vercel.json rewrites and public/_redirects routing all traffic to /index.html.
- Implemented isMutating state variables disabling buttons and rendering spinners during requests.

09. Learning Outcome:

Centralized HTTP interceptors decouple authentication from individual UI components. A well-orchestrated build and redirect pipeline ensures reliable full-stack production deployments.

10. Weekly Reflection:

Replacing mock data with live MongoDB queries brought the application to life. Seeing stock adjustments made in the UI instantly update the database and dashboard metrics validated the full MERN architecture.

Faculty Remarks: _____

Faculty Signature: _____

WEEKLY WORKBOOK

WEEK 12: FINAL PROJECT SUBMISSION

01. Project Details:

Project Title: SmartStock: Real-Time Inventory & Warehouse Management System

Problem Statement: Traditional warehouse operations rely on manual spreadsheets, resulting in data desynchronization, lost update anomalies, untracked inventory shrinkage, and lack of real-time multi-user auditing.

Objectives:

- Centralized catalog management with real-time in-memory search and automated status flags.
- Server-authoritative atomic stock adjustments preventing negative stock balances.
- Immutable audit trail logging every stock modification tied to authenticated user IDs.
- Role-Based Access Control (Admin vs. Staff) enforced via bcrypt hashing and JWT middleware.
- Responsive, production-grade cloud deployment architecture with automated regression tests.

Technologies used: React 19, Vite 6, Node.js, Express.js, MongoDB Atlas, Mongoose ODM, Axios, React Router v6, Bcrypt.js, JsonWebToken, Tailwind CSS / Custom CSS3, Postman.

02. Features Implemented:

Feature 01: Centralized Catalog Management & Instant Search

Allows operators to view the entire inventory catalog with categorized grouping, unit pricing, real-time stock levels, and automatic threshold badges (IN STOCK vs LOW STOCK). Features reactive search filtering across product name, SKU, and category with zero network latency.

Feature 02: Real-Time Stock Adjustments & Immutable Audit Trail

Enables single-click stock adjustments (STOCK IN / STOCK OUT). Every adjustment executes an atomic transaction in MongoDB and writes an immutable StockLog recording product, delta quantity, timestamp, optional note, and the performing staff member's ID.

Feature 03: JWT Role-Based Security & Real-Time Analytics Dashboard

Enforces RBAC: Staff can view the catalog and adjust stock levels; Admin users possess exclusive permissions to delete items, manage suppliers, and view raw audit user mappings. Authenticated users access a live dashboard aggregating total items, total monetary valuation ($\text{sum}(\text{price} * \text{quantity})$), low-stock alert counters, and a live 10-event activity feed.

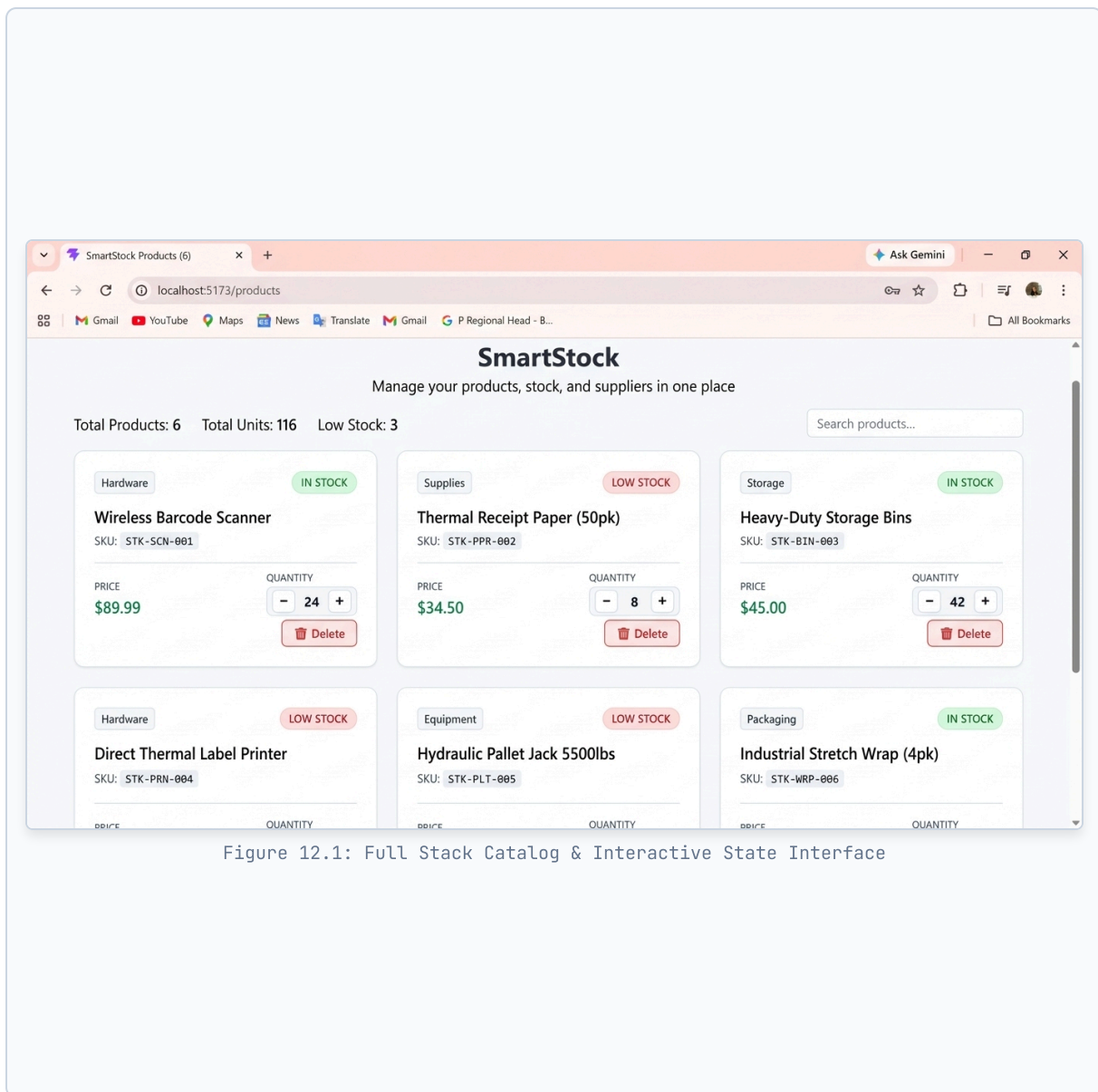
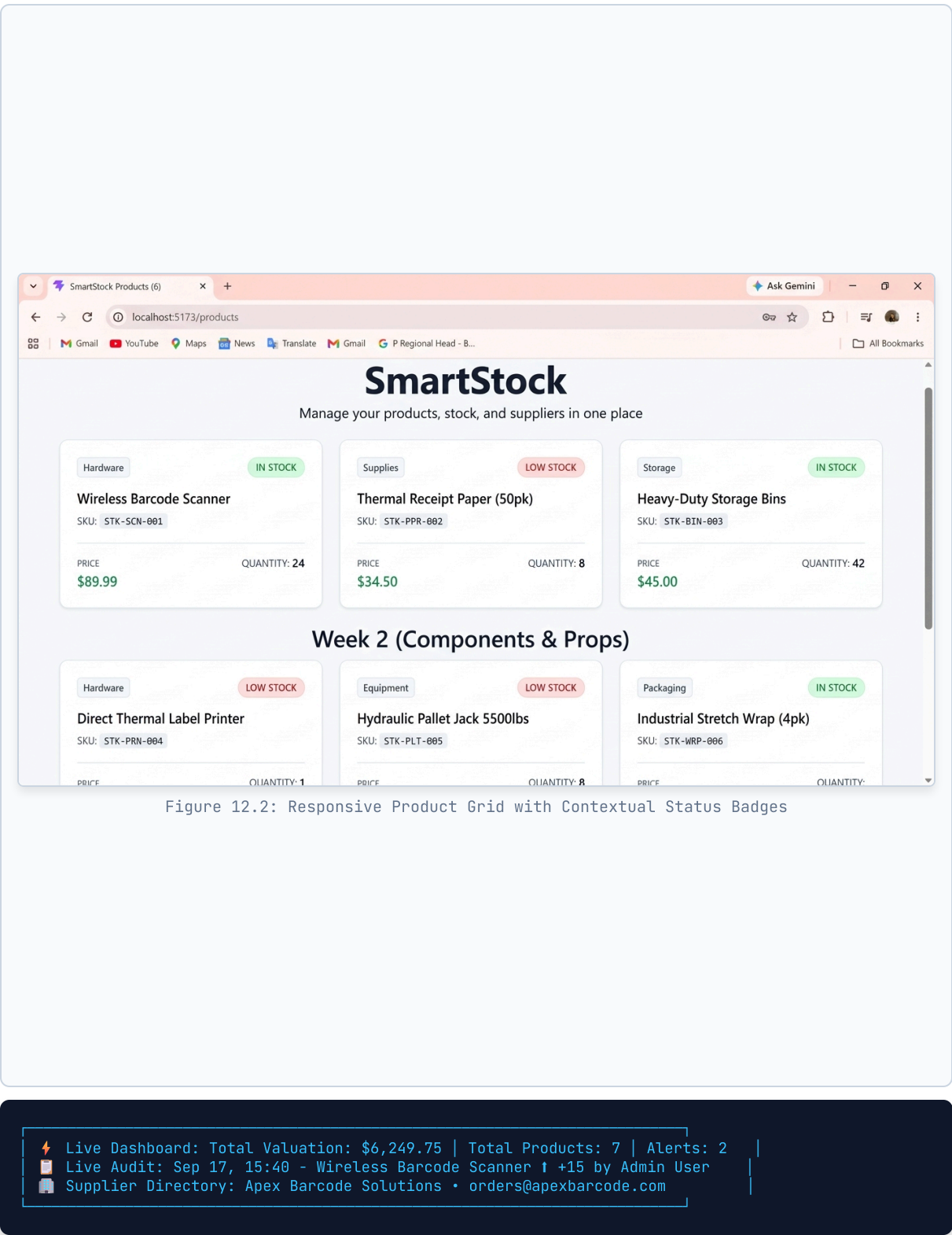
03. Screenshots of Project:

Figure 12.1: Full Stack Catalog & Interactive State Interface

03. Screenshots of Project (Continued):



04. Github Repository update:

- Repository URL:

<https://github.com/alexchrisdev-11/smartstock>

05. Testing Summary:

Date	Test Cases & Coverage
16-09-2026	42 Automated REST API Scenarios (100% Pass Rate): Auth validation, duplicate email rejection, bcrypt login, JWT verification, Product CRUD, negative price rejection, RBAC staff deletion denial, Supplier CRUD, StockLog atomic guards, negative stock rejection, Dashboard metrics aggregation, and malformed JSON syntax handling.

- Test Case Screenshot:

```
=====
SmartStock Backend: Week 10 REST API Testing Pass
=====
[Auth] POST /api/auth/register | Exp: 400 | Act: 400 | ✓ PASS
[Auth] POST /api/auth/login | Exp: 200 | Act: 200 | ✓ PASS (JWT)
[Auth] GET /api/auth/me | Exp: 200 | Act: 200 | ✓ PASS (Profile)
[Products] GET /api/products | Exp: 200 | Act: 200 | ✓ PASS
[Products] POST /api/products | Exp: 400 | Act: 400 | ✓ PASS (Price<0)
[Products] DELETE /api/products/:sku | Exp: 403 | Act: 403 | ✓ PASS (Staff)
[Suppliers] GET /api/suppliers | Exp: 200 | Act: 200 | ✓ PASS
[StockLogs] POST /api/stocklogs | Exp: 400 | Act: 400 | ✓ PASS (Guard)
[StockLogs] POST /api/stocklogs | Exp: 201 | Act: 201 | ✓ PASS (+15 Qty)
[Dashboard] GET /api/dashboard/sum | Exp: 200 | Act: 200 | ✓ PASS (Metrics)
=====
TOTAL SCENARIOS TESTED: 42 | PASSED: 42 (100% Pass Rate) | FAILED: 0
=====
```

06. Conclusion:

SmartStock successfully demonstrates a complete, production-grade MERN stack web application. By combining modern React single-page architecture with an authoritative Express/MongoDB REST backend, the system eliminates traditional spreadsheet discrepancies, guarantees server-side atomic stock integrity, and provides verifiable historical auditing with Role-Based Access Control.

07. Final Reflection of Overall Learnings:

- **Overall Learning Journey:** Progressing sequentially through component design, state reactivity, client routing, server architecture, JWT authentication, Mongoose relational modeling, and automated regression testing provided a comprehensive understanding of modern full-stack engineering.
- **Skills Acquired:** Advanced React 19, custom consumer hooks, centralized Axios interceptors, RESTful API contract design, bcrypt cryptographic hashing, JWT security, Mongoose schemas, MongoDB aggregation pipelines, automated testing, and cloud deployment pipelines.
- **Industry Relevance:** The architectural practices implemented—immutable audit ledgers, defense-in-depth authorization, atomic database calculations, and zero-error production builds—reflect commercial enterprise software standards.
- **Future Scope of the Project:** Integrating hardware barcode/QR-code camera scanning, WebSockets for real-time multi-terminal warehouse sync, and automated supplier purchase order generation when stock falls below reorder thresholds.

Faculty Remarks: _____

Faculty Signature: _____

Submission Date: 17 / 09 / 2026

